

TECHNICAL INTERVIEW PREPARATION

AWS

Cloud Engineer & DevOps Engineer

50 Expert-Level Interview Questions with Full Answer Guides

EC2 · ALB · Auto Scaling · RDS · S3 · Lambda · Route 53 · CloudFront

VPC · Security Groups · NACLs · EKS · IAM · CloudWatch · SQS/SNS · IaC

Total Questions	Difficulty Mix	Services Covered	Target Audience	Version
50 Questions	Easy: 0 Medium: 25 Hard: 25	16 AWS Services + Cross-Service	Cloud Engineer, DevOps Engineer, SRE	March 2026

How to Use This Guide

Each question card contains:

What they're testing — the underlying skill or concept being assessed

Key Answer Points — 6-8 bullet points that a senior engineer would cover

Follow-up Probe — the likely second question if you answer the first one well

Difficulty: **Medium** = mid-level engineer expected to know this. **Hard** = senior/staff engineer answer expected.

TABLE OF CONTENTS

S e c t i o n	AWS Service / Topic	Questions	Count
1	EC2 — Elastic Compute Cloud	Q1–Q5	5
2	ALB — Application Load Balancer	Q6–Q10	5
3	Auto Scaling Groups	Q11–Q14	4
4	RDS — Relational Database Service	Q15–Q19	5
5	S3 — Simple Storage Service	Q20–Q23	4
6	Lambda — Serverless Compute	Q24–Q27	4
7	Route 53 — DNS & Traffic Routing	Q28–Q30	3
8	CloudFront — CDN & Edge	Q31–Q32	2
9	VPC — Virtual Private Cloud	Q33–Q35	3
10	Security Groups & NACLs	Q36–Q37	2
11	EKS — Elastic Kubernetes Service	Q38–Q40	3
12	IAM & Security	Q41–Q42	2
13	CloudWatch & Observability	Q43–Q44	2
14	Cost Optimisation	Q45–Q46	2
15	DevOps, IaC & CI/CD (Terraform, CodePipeline)	Q47–Q48	2
16	SQS / SNS / EventBridge / Secrets Manager	Q49–Q50	2



EC2 — Elastic Compute Cloud

Q1–Q5 · Interview Questions

Q1



EC2

Medium

Your EC2 application suddenly starts failing ALB health checks. Walk me through your full troubleshooting process step by step.

What they're testing: Systematic troubleshooting, OSI-layer thinking, production calmness

Key Answer Points:

- ▶ Check ALB Target Group — are targets Unhealthy or Draining?
- ▶ SSH to instance and test the health check endpoint locally: `curl -sv http://localhost:<port><path>`
- ▶ Verify Security Group allows ALB's SG on the health check port (inbound)
- ▶ Check application logs for crashes, OOM, or handler errors
- ▶ Verify the process is running: `systemctl status <app>` and `ss -tlnp`
- ▶ Check instance CPU/memory — if overloaded, app may be too slow to respond within timeout
- ▶ Review ALB access logs for HTTP status codes returned to the health checker
- ▶ If intermittent: check for auto-scaling launch of bad AMI or a failing deploy

Follow-up Probe: How would you set up an automated CloudWatch alarm to detect ALB health check failures before users notice?

Q2



EC2

Hard

An EC2 instance in a private subnet needs to download packages from the internet but has no direct internet access. What are all the ways to enable this, and when would you choose each?

What they're testing: Networking architecture, NAT options, cost-vs-security trade-offs

Key Answer Points:

- ▶ NAT Gateway (managed, HA, per-GB cost) — best for production, scales automatically
- ▶ NAT Instance (EC2 running NAT AMI) — cheaper, you manage HA, good for dev/low-traffic
- ▶ VPC Endpoints (Gateway for S3/DynamoDB, Interface for most AWS services) — no internet needed at all
- ▶ AWS PrivateLink — for third-party SaaS services privately
- ▶ Proxy (Squid on EC2) — when you need URL filtering / egress control
- ▶ Session Manager + internet access from bastion — for one-off commands only

Follow-up Probe: If cost is the primary concern and traffic is low, how would you replace a NAT Gateway without impacting availability?

Q3



EC2

Medium

You need to run a startup script on a new EC2 instance every time it launches (even after stop/start). How do you implement this, and what are the pitfalls?

What they're testing: EC2 user data, cloud-init, instance lifecycle

 Key Answer Points:

- ▶ User data runs on FIRST launch only by default — must use cloud-init with 'always' frequency to run on every start
- ▶ Use: #cloud-config with runcmd and frequency: always in /var/lib/cloud/scripts/per-always/
- ▶ Alternative: systemd service that runs the script at boot (more reliable, easier to debug)
- ▶ Pitfall: User data size limit is 16KB — use S3 to store scripts and download in user data
- ▶ Pitfall: Errors in user data silently fail — always write logs to /var/log/user-data.log
- ▶ Debug: Check /var/log/cloud-init-output.log for execution details

 **Follow-up Probe:** How would you pass secrets (DB passwords) into a launch script securely without hardcoding them in user data?

Q4

 EC2

Hard

Your team is unsure whether to use On-Demand, Reserved, Savings Plans, or Spot Instances for a new workload. How do you decide?

 **What they're testing:** Cost optimisation, instance purchasing strategy, workload analysis

 Key Answer Points:

- ▶ On-Demand: unpredictable workloads, spikes, dev/test — pay per hour, no commitment
- ▶ Reserved Instances (1yr/3yr): steady-state predictable workloads — up to 72% saving
- ▶ Savings Plans: flexible — covers EC2, Lambda, Fargate across families — up to 66% saving
- ▶ Spot: fault-tolerant, stateless, batch jobs — up to 90% saving but can be reclaimed with 2-min notice
- ▶ Decision framework: baseline load → Savings Plans or RIs; variable → On-Demand; batch/CI → Spot
- ▶ Use Spot with Spot Fleet + capacity-optimised allocation strategy for resilience
- ▶ Tools: AWS Cost Explorer, Compute Optimizer, Spot Instance Advisor

 **Follow-up Probe:** How would you architect a batch processing pipeline that uses Spot Instances but guarantees completion even when Spot is reclaimed?

Q5

 EC2

Medium

A production EC2 instance is experiencing high CPU usage intermittently. How do you investigate and fix it without causing downtime?

 **What they're testing:** Performance troubleshooting, CloudWatch metrics, non-disruptive debugging

 Key Answer Points:

- ▶ CloudWatch: CPU Utilisation metric — is it consistent or spiky? Set alarm at 80%
- ▶ SSH and run: top, htop, ps aux --sort=-%cpu — identify the process
- ▶ Check for runaway processes, cron jobs, scheduled tasks
- ▶ Check application metrics — are specific endpoints causing CPU spikes?
- ▶ Enable CloudWatch detailed monitoring (1-min granularity) for better correlation
- ▶ Consider vertical scale (larger instance type via stop/start) or horizontal (add to ASG)
- ▶ Profile the application code — CPU spikes may indicate algorithmic issues
- ▶ Examine CloudWatch Logs Insights for correlation between errors and CPU peaks

 **Follow-up Probe:** How would you implement CPU-based auto-scaling that scales out before CPU hits 100% and scales in gradually to avoid thrashing?



ALB — Application Load Balancer

Q6–Q10 · Interview Questions

Q6

 ALB

Medium

Your ALB is returning 502 Bad Gateway errors for some requests but not all. How do you systematically identify which targets are causing it?

 **What they're testing:** ALB troubleshooting, target health, access logs analysis

 Key Answer Points:

- ▶ Enable ALB Access Logs to S3 — query with Athena: find all 502s, correlate target_ip
- ▶ Check Target Group health — some targets may be Unhealthy while others serve traffic
- ▶ 502 from ALB means: target returned invalid HTTP response, connection refused, or timed out
- ▶ Check response timeout in Target Group settings vs actual app response time
- ▶ SSH to specific target and test the endpoint directly to reproduce locally
- ▶ Check application logs for HTTP handler panics or malformed responses
- ▶ Review CloudWatch Metrics: HTTPCode_Target_5XX_Count per target

 **Follow-up Probe:** How would you configure ALB to route traffic away from an unhealthy target faster — what settings would you tune?

Q7

 ALB

Hard

You need to route requests to different microservices based on URL path and headers, with one service requiring mutual TLS. How do you design this ALB configuration?

 **What they're testing:** Advanced ALB routing, listener rules, mTLS, certificate management

 Key Answer Points:

- ▶ ALB supports path-based routing: /api/* → Target Group A, /web/* → Target Group B
- ▶ Header-based routing: Host header → different services (one ALB, multiple domains)
- ▶ Listener rules: priority order matters — most specific rules first
- ▶ For mTLS: use ALB mutual authentication (AWS Certificate Manager integration) — requires client cert
- ▶ Alternative for mTLS: terminate at service level using NLB passthrough + application-level mTLS
- ▶ Configure separate Target Groups per service for independent health checking
- ▶ Use weighted target groups for canary deployments (10% new, 90% old)

 **Follow-up Probe:** How would you implement A/B testing at the ALB level without changing application code?

Q8

 ALB

Medium

A client complains that their session is being dropped every few minutes on your ALB. What are the possible causes and fixes?

 **What they're testing:** Session persistence, sticky sessions, connection management

 Key Answer Points:

- ▶ Check if sticky sessions (session affinity) is enabled on the Target Group
- ▶ ALB sticky sessions use a cookie (AWSALB) with configurable duration
- ▶ If duration < session length, users get re-routed — extend the stickiness duration
- ▶ Check if Target Group is replacing instances (ASG scale-in removes sticky target)
- ▶ Idle timeout: ALB default is 60s — if requests are less frequent, connection drops
- ▶ Increase idle timeout: AWS Console → ALB Attributes → Idle Timeout
- ▶ Better fix: store session in ElastiCache/Redis so any instance can serve the session

 **Follow-up Probe:** Why is sticky session at the load balancer layer considered an anti-pattern, and what is the preferred architecture?

Q9

 ALB

Hard

How would you protect your ALB-fronted application from a DDoS attack and common web exploits like SQL injection and XSS?

 **What they're testing:** Security architecture, WAF, Shield, rate limiting

 Key Answer Points:

- ▶ AWS WAF on ALB: managed rule groups (SQL injection, XSS, known bad IPs, OWASP Top 10)
- ▶ AWS Shield Standard: automatic Layer 3/4 DDoS protection (free on ALB)
- ▶ AWS Shield Advanced: Layer 7 DDoS protection with 24/7 DDoS response team (paid)

- ▶ Rate-based rules in WAF: block IPs sending >1000 req/5min
- ▶ Geo-restriction in WAF: block or allow specific countries
- ▶ CloudFront in front of ALB: absorbs volumetric attacks at edge before reaching origin
- ▶ ALB + WAF + Shield + CloudFront = defence in depth for web applications

💬 **Follow-up Probe:** How would you handle a DDoS attack that is specifically targeting your application at Layer 7 (HTTP floods)?

Q10

 ALB

Medium

You need to implement a blue-green deployment using ALB with zero downtime. Describe the complete strategy.

 **What they're testing:** Deployment strategies, traffic shifting, rollback

 Key Answer Points:

- ▶ Create two Target Groups: blue (current) and green (new version)
- ▶ Deploy new version to green Target Group, run health checks and smoke tests
- ▶ Use ALB weighted routing: shift traffic gradually (10% → 50% → 100%) to green
- ▶ Monitor: error rates, latency, business metrics during traffic shift
- ▶ If metrics are healthy: shift 100% to green, deregister blue targets
- ▶ If issues detected: shift 100% back to blue instantly (single API call)
- ▶ Can also use Route 53 weighted records for multi-region blue-green
- ▶ AWS CodeDeploy natively supports ALB blue/green deployments with automatic rollback

 **Follow-up Probe:** What is the difference between blue-green and canary deployments, and when would you choose one over the other?



Auto Scaling Groups

Q11–Q14 · Interview Questions

Q11

 Auto Scaling

Hard

Your Auto Scaling Group is constantly scaling in and out (thrashing) causing instability. What is causing this and how do you fix it?

 **What they're testing:** ASG tuning, scale-in protection, cooldown periods, metric selection

 Key Answer Points:

- ▶ Thrashing: scaling out triggers immediately, then scale-in fires before new instances are needed again
- ▶ Fix 1: Increase cooldown period (default 300s) — prevents back-to-back scaling actions
- ▶ Fix 2: Use step scaling or target tracking with longer scale-in cooldown than scale-out
- ▶ Fix 3: Set scale-in protection on recently launched instances
- ▶ Fix 4: Change metric — CPU-based can thrash; use SQS queue depth or custom metric
- ▶ Fix 5: Set instance warmup period — new instances not counted in aggregate metrics during warmup
- ▶ Target Tracking Policy: self-adjusts and has built-in stabilisation — prefer over step scaling

 **Follow-up Probe:** How would you implement predictive scaling for a workload with known traffic patterns (e.g., business hours spike)?

Q12

 Auto Scaling

Medium

An instance in your ASG is being terminated but it is still processing a critical long-running job. How do you protect it?

 **What they're testing:** Lifecycle hooks, graceful termination, workload protection

 Key Answer Points:

- ▶ ASG Lifecycle Hooks: intercept termination with a Terminating:Wait hook
- ▶ In the hook: send a heartbeat signal or complete the job then call complete-lifecycle-action
- ▶ Default hook timeout is 1 hour — configurable up to 48 hours
- ▶ Implement hook handler: Lambda triggered by EventBridge on EC2 lifecycle event
- ▶ Handler logic: check if job is running → if yes, send heartbeat → when done, complete hook
- ▶ Instance scale-in protection: set programmatically when job starts, remove when job finishes
- ▶ Never rely on just terminationGracePeriodSeconds for long jobs

 **Follow-up Probe:** How would you implement a graceful drain process that finishes all in-flight requests before an ASG terminates an ALB-registered instance?

Q13

 Auto Scaling

Hard

Design an auto-scaling strategy for a stateful application (e.g., gaming server) where you cannot simply terminate any instance.

 **What they're testing:** Stateful ASG design, instance drain, custom termination policies

 Key Answer Points:

- ▶ Use custom termination policy to control WHICH instance gets terminated
- ▶ Tag instances with status (busy/idle) — custom policy only terminates 'idle' instances
- ▶ Scale-in protection: programmatically protect busy instances via AWS SDK
- ▶ Use ASG Lifecycle Hooks to drain the instance (move players to other servers) before termination
- ▶ Consider separate ASGs for different game states
- ▶ NLB (not ALB) with connection draining for stateful TCP connections
- ▶ Use EventBridge + Lambda to react to scale-in events and orchestrate migration

 **Follow-up Probe:** How would you handle the case where all instances are busy and the ASG wants to scale in due to low CPU but traffic could spike again?

Q14

 Auto Scaling

Medium

Your ASG is launching new instances on scale-out events but they are failing health checks and being terminated before serving traffic. How do you debug this?

 **What they're testing:** Launch configuration debugging, health check configuration, instance troubleshooting

 Key Answer Points:

- ▶ Check ASG activity history — it shows exact reason for termination
- ▶ Check EC2 system status checks and instance status checks for the failed instances
- ▶ Review user data / launch template — startup script may be failing silently
- ▶ Connect to a failed instance via SSM Session Manager before it is terminated
- ▶ Use ASG instance refresh with mixed instance behaviour to catch issues
- ▶ Add an EC2 health check grace period (default 300s) — gives app time to start before health check runs
- ▶ Check AMI — if AMI is broken, every new instance will fail the same way

 **Follow-up Probe:** How would you set up a canary launch — testing new AMIs on 1-2 instances before rolling to the full ASG?



RDS — Relational Database Service

Q15–Q19 · Interview Questions

Q15

 RDS

Hard

Your production RDS database is approaching storage limits and will run out within hours. What do you do right now?

 **What they're testing:** RDS storage management, emergency response, autoscaling storage

 Key Answer Points:

- ▶ IMMEDIATE: Enable Storage Autoscaling in RDS console — it auto-expands when 10% free space remains
- ▶ If not enabled: manually increase allocated storage (no downtime for most engines in Multi-AZ)
- ▶ Run on DB: identify top tables by size, check for runaway log tables or temp data
- ▶ Check for large transaction logs or undo logs (MySQL/Postgres) consuming space
- ▶ Enable automated backups review — backup retention consuming space on DB instance
- ▶ Set CloudWatch alarm on FreeStorageSpace metric: alert at 20%, critical at 10%
- ▶ Long-term: archive old data to S3, implement data lifecycle policies

 **Follow-up Probe:** How would you implement proactive storage monitoring to prevent this from happening again, and what threshold would you alert on?

Q16

RDS

Hard

Your RDS read replica is lagging several minutes behind the primary. How do you diagnose and resolve this, and what are the implications?

🔗 **What they're testing:** RDS replication lag, read replica architecture, performance tuning

✅ **Key Answer Points:**

- ▶ Check CloudWatch: ReplicaLag metric — lag in seconds
- ▶ Causes: heavy write load on primary, long-running transactions, low replica instance class
- ▶ Diagnosis: check binlog position gap, slow query log on replica, replica instance CPU/IO
- ▶ Fix 1: Upgrade replica instance class to match or exceed primary
- ▶ Fix 2: Reduce write load on primary — batch writes, avoid long transactions
- ▶ Fix 3: Enable Multi-AZ on primary (sync replication) for HA; read replicas are async
- ▶ Fix 4: Consider Aurora — its distributed storage eliminates traditional replication lag
- ▶ Implication: apps reading from replica may get stale data — add staleness tolerance in app

💬 **Follow-up Probe:** How would you architect an application that tolerates some read replica lag without showing stale data to users?

Q17

RDS

Medium

A developer accidentally dropped a production table in RDS. How do you recover it as quickly as possible?

🔗 **What they're testing:** RDS backup/restore, point-in-time recovery, disaster recovery

✅ **Key Answer Points:**

- ▶ Option 1 (fastest): Restore to a new RDS instance using Point-In-Time Recovery (PITR) to 5 minutes before the drop
- ▶ PITR is available if automated backups are enabled (always enable in production)
- ▶ Restore creates a NEW instance — export the dropped table and import into production
- ▶ Option 2: Restore from latest automated snapshot (less granular than PITR)
- ▶ Option 3: If using Aurora, use Aurora Backtrack to rewind the cluster in-place without a new instance
- ▶ Never restore directly to production — always restore to a temporary instance first
- ▶ Post-incident: implement database activity streams and query approval for DROP/TRUNCATE

💬 **Follow-up Probe:** How would you implement a 'break-glass' procedure that requires approval before destructive SQL commands (DROP, TRUNCATE) can run in production?

Q18

RDS

Hard

Your application is experiencing connection pool exhaustion on RDS. Describe the root causes and solutions.

🔗 **What they're testing:** RDS connection management, RDS Proxy, connection pooling

✅ **Key Answer Points:**

- ▶ Root causes: too many app instances each opening many connections; connection leaks; long transactions holding connections

- ▶ RDS has a max_connections limit based on instance class (RAM-based formula)
- ▶ Solution 1: RDS Proxy — pools and multiplexes connections; reduces actual DB connections by up to 90%
- ▶ RDS Proxy also handles failover transparently (cuts from 30s to <30s)
- ▶ Solution 2: Implement connection pooling in app (PgBouncer for PostgreSQL, ProxySQL for MySQL)
- ▶ Solution 3: Tune max_connections parameter group upward (may need instance upsize)
- ▶ Monitor: DatabaseConnections CloudWatch metric, set alarm at 80% of max_connections
- ▶ Connection leaks: enable connection idle timeout; use connection lifecycle tracking

💬 **Follow-up Probe:** How does RDS Proxy handle a Multi-AZ failover differently from a direct RDS connection, and why does this matter for Lambda functions?

Q19

 RDS

Medium

How would you migrate a 5TB on-premise Oracle database to Amazon Aurora PostgreSQL with minimal downtime?

 **What they're testing:** Database migration strategy, DMS, heterogeneous migration

 Key Answer Points:

- ▶ Use AWS DMS (Database Migration Service) for continuous replication
- ▶ Phase 1: Schema conversion with AWS SCT (Schema Conversion Tool) — converts Oracle schema to PostgreSQL
- ▶ Phase 2: Full load — DMS migrates all existing data to Aurora
- ▶ Phase 3: CDC (Change Data Capture) — DMS continuously replicates changes during cutover window
- ▶ Phase 4: Cutover — stop writes to Oracle, wait for DMS lag to reach 0, switch app connection string
- ▶ Test migration: perform full rehearsal in dev/staging first — measure actual cutover time
- ▶ Validate: row counts, checksums, query results between source and target
- ▶ Downtime window: only the final cutover (minutes, not hours)

 **Follow-up Probe:** What are the most common gotchas when converting from Oracle to PostgreSQL with AWS SCT, and how do you handle stored procedures?



S3 — Simple Storage Service

Q20–Q23 · Interview Questions

Q20

 S3

Medium

An S3 bucket with critical data was accidentally deleted. What are your options to recover the data?

 **What they're testing:** S3 versioning, cross-region replication, backup strategy

 Key Answer Points:

- ▶ If versioning was enabled: objects can be recovered by deleting the delete marker
- ▶ If bucket itself deleted: if versioning was on, the bucket contents are gone with the bucket — you must restore from backup
- ▶ If S3 Cross-Region Replication (CRR) was configured: replicated bucket has all objects
- ▶ If S3 Replication Time Control (RTC): guaranteed replication within 15 minutes
- ▶ If AWS Backup was configured for S3: restore from backup vault
- ▶ Lesson: enable versioning + MFA delete on all critical buckets; enable CRR; use S3 Object Lock
- ▶ S3 Object Lock (WORM): prevents ANY deletion or overwrite for compliance retention periods

 **Follow-up Probe:** How would you implement a bucket policy that prevents deletion of objects even by the bucket owner, and what AWS feature enables this?

Q21

 S3

Hard

You need to serve private S3 objects to authenticated users via CloudFront without making the bucket public. How do you architect this?

🎯 **What they're testing:** S3 OAC/OAI, CloudFront signed URLs, bucket policies

✅ **Key Answer Points:**

- ▶ Use CloudFront Origin Access Control (OAC) — newer and more secure than legacy OAI
- ▶ OAC: CloudFront gets an identity that is granted GetObject permission on the S3 bucket
- ▶ Bucket policy: only allow s3:GetObject from the CloudFront OAC principal
- ▶ Block all public access on the bucket — objects only accessible via CloudFront
- ▶ For per-user access: use CloudFront Signed URLs or Signed Cookies (Lambda@Edge generates them)
- ▶ Signed URL: time-limited, user-specific URL that CloudFront validates
- ▶ Signed Cookie: better for streaming — user authenticates once, can access multiple objects
- ▶ CloudFront Function or Lambda@Edge handles authentication before forwarding to S3

💬 **Follow-up Probe:** How would you implement a signed URL system where URLs expire after 15 minutes and each URL is tied to a specific user's account?

Q22



Medium

Your S3 data lake is growing rapidly and costs are increasing. How would you implement an intelligent cost optimisation strategy?

What they're testing: S3 storage classes, lifecycle policies, cost optimisation

Key Answer Points:

- ▶ S3 Intelligent-Tiering: automatically moves objects between frequent/infrequent tiers — no retrieval fee
- ▶ S3 Lifecycle Policies: transition objects based on age
 - ▶ Standard → Standard-IA (after 30 days) → Glacier Instant Retrieval (after 90 days) → Glacier Deep Archive (after 180 days)
- ▶ Use S3 Storage Class Analysis to find objects eligible for transition
- ▶ Enable S3 Inventory to audit what's in the bucket and identify old/large objects
- ▶ Multipart Upload cleanup: abort incomplete uploads with lifecycle rule
- ▶ Enable S3 storage lens for organisation-wide visibility of bucket usage
- ▶ Compress data before storing (Parquet + Snappy/ZSTD for analytics data)

Follow-up Probe: How would you estimate the cost savings before implementing S3 Intelligent-Tiering vs a manual lifecycle policy for your specific access patterns?

Q23



Hard

You need to process objects as soon as they are uploaded to S3, but your processing Lambda is being overwhelmed by a large burst of uploads. How do you design a resilient solution?

What they're testing: S3 event notifications, SQS, Lambda event source mapping, fan-out

Key Answer Points:

- ▶ S3 → SQS (not directly to Lambda) — SQS buffers the burst and provides back-pressure
- ▶ Lambda polls SQS as event source: scales concurrency based on queue depth
- ▶ Set SQS visibility timeout > Lambda max execution time to prevent duplicate processing
- ▶ Configure Lambda concurrency limit to protect downstream resources from Lambda bursts
- ▶ Dead Letter Queue (DLQ) on SQS: capture failed processing events for retry/debugging
- ▶ S3 Event Notifications: configure prefix/suffix filters to only trigger for relevant objects
- ▶ For very high throughput: SNS fanout → multiple SQS queues for parallel processing pipelines
- ▶ Monitor: SQS ApproximateNumberOfMessages and Lambda Throttles metrics

Follow-up Probe: How would you implement exactly-once processing guarantee for S3 objects when multiple Lambda instances may process the same S3 event?

λ Lambda — Serverless Compute

Q24–Q27 · Interview Questions

Q24



Medium

Your Lambda function is timing out frequently. Walk me through a systematic troubleshooting and optimisation approach.

🎯 **What they're testing:** Lambda optimisation, cold starts, execution time, profiling

✅ **Key Answer Points:**

- ▶ Check CloudWatch Logs: is it always timing out or intermittently? Pattern indicates cause.
- ▶ Check what the function is doing near timeout: add structured logging with timestamps
- ▶ Common causes: external API/DB slow response, cold start, insufficient memory, large payload
- ▶ Cold start: Lambda initialises container — reduce with Provisioned Concurrency or SnapStart (Java)
- ▶ Increase timeout (max 15 min) as immediate fix, but understand WHY it's slow
- ▶ Increase memory: Lambda CPU scales with memory — often 512MB → 1024MB halves execution time
- ▶ Connection pooling: initialise DB connections outside the handler (in init code) to reuse across invocations
- ▶ AWS Lambda Power Tuning: open-source tool that finds optimal memory/cost configuration

💬 **Follow-up Probe:** What is the difference between Lambda Provisioned Concurrency and Lambda SnapStart, and which would you use for a Java function that must respond in under 100ms?

Q25 **λ Lambda****Hard**

Design a serverless architecture that processes 10 million events per day with guaranteed delivery, exactly-once processing semantics, and the ability to replay failed events.

 **What they're testing:** Serverless architecture, event sourcing, idempotency, replay

 Key Answer Points:

- ▶ Event ingestion: API Gateway + SQS FIFO (ordering + deduplication) or Kinesis Data Streams
- ▶ Exactly-once: SQS FIFO MessageDeduplicationId + Lambda idempotency key stored in DynamoDB
- ▶ Processing: Lambda with SQS event source mapping — batch size 10, retry with backoff
- ▶ DLQ: failed events go to DLQ after max attempts — alert and process separately
- ▶ Replay: store all raw events in S3 (event store). Replay by re-processing from S3.
- ▶ EventBridge Pipes: orchestrate filter → enrich → deliver pipeline without code
- ▶ DynamoDB TTL on idempotency keys: auto-clean after replay window (e.g., 7 days)
- ▶ Monitoring: custom CloudWatch metric for processing lag, error rate, DLQ depth

 **Follow-up Probe:** How would you handle a scenario where your Lambda has processed an event and written to DynamoDB, but the SQS message visibility timeout expired and the same event is delivered again?

Q26 **λ Lambda****Medium**

Your Lambda function needs to access an RDS database in a private subnet and also call an external public API. How do you configure this?

 **What they're testing:** Lambda VPC configuration, NAT Gateway, security groups

 Key Answer Points:

- ▶ Put Lambda in the VPC to access private RDS — configure VPC, subnets (private), security group
- ▶ Lambda SG: outbound to RDS SG on port 5432/3306
- ▶ RDS SG: inbound from Lambda SG
- ▶ For external API access: Lambda in private subnet needs NAT Gateway in public subnet
- ▶ Lambda execution role needs `ec2:CreateNetworkInterface`, `DescribeNetworkInterfaces`, `DeleteNetworkInterface`
- ▶ VPC Lambda has ~1s additional cold start due to ENI allocation — mitigate with Provisioned Concurrency
- ▶ Alternative for RDS: use RDS Proxy (no VPC Lambda needed for connection pooling benefits)
- ▶ For AWS services (S3, DynamoDB): use VPC Endpoints to avoid NAT Gateway costs

 **Follow-up Probe:** What is the impact on Lambda scalability when you put it inside a VPC, and how does AWS Hyperplane address this limitation?

Q27 **λ Lambda****Hard**

How do you securely store and access secrets (API keys, DB passwords) in Lambda functions across multiple environments (dev, staging, prod)?

 **What they're testing:** Secrets management, IAM, AWS Secrets Manager, Parameter Store

 Key Answer Points:

- ▶ AWS Secrets Manager: store secrets, automatic rotation, versioning — best for DB credentials
- ▶ Parameter Store (SSM): cheaper, free for Standard tier, supports SecureString (KMS encrypted)
- ▶ Lambda IAM role: grant secretsmanager:GetSecretValue or ssm:GetParameter to specific ARNs only
- ▶ Retrieve at init time (outside handler): cached in Lambda execution environment between invocations
- ▶ Never hardcode secrets in code or environment variables (they appear in console/logs)
- ▶ Use environment-specific secret paths: /myapp/prod/db-password vs /myapp/dev/db-password
- ▶ Secrets rotation: Secrets Manager rotates RDS credentials automatically; Lambda gets latest on next init
- ▶ Lambda Extension for Secrets Manager: caches secrets locally, refreshes before expiry

💬 **Follow-up Probe:** How would you implement secret rotation without any downtime when the Lambda function is invoked thousands of times per minute?



Route 53 — DNS & Traffic Routing

Q28–Q30 · Interview Questions

Q28

Route 53

Hard

You need to implement multi-region active-active architecture with intelligent failover. How do you design the Route 53 configuration?

What they're testing: Route 53 routing policies, health checks, multi-region architecture

Key Answer Points:

- ▶ Use Latency-Based Routing: routes users to the lowest-latency region automatically
- ▶ Health Checks: configure for each regional endpoint — monitors ALB, NLB, or custom endpoint
- ▶ Failover Policy: primary + secondary; Route 53 automatically removes unhealthy endpoints
- ▶ Combined: Latency routing with health check failover — fastest healthy region wins
- ▶ Global Accelerator: alternative to Route 53 for TCP/UDP with static anycast IPs
- ▶ Route 53 Application Recovery Controller: readiness checks + routing controls for complex failovers
- ▶ TTL: set low (60s) for records that may need to fail over — but be aware of DNS caching
- ▶ Test: use Route 53 health check + synthetic monitor to simulate regional failures

Follow-up Probe: How does Route 53 Application Recovery Controller differ from standard Route 53 health check failover, and when would you need it?

Q29

Route 53

Medium

A DNS change you made in Route 53 is not propagating to all clients even after 24 hours. How do you troubleshoot this?

What they're testing: DNS TTL, propagation, caching, dig/nslookup debugging

Key Answer Points:

- ▶ Check the actual TTL on the record — was it 86400 (24h)? Old TTL persists until it expires
- ▶ Use dig to query Route 53 directly: `dig @8.8.8.8 <domain>` — bypasses local cache
- ▶ Check resolver caches: corporate DNS, ISP DNS, browser DNS cache (`chrome://net-internals/#dns`)
- ▶ Negative caching: if domain previously returned NXDOMAIN, negative TTL may be caching that
- ▶ Verify the change is actually applied in Route 53 console — it may not have been saved
- ▶ Use 'dig +trace' to follow the full resolution chain and find where the stale answer is coming from
- ▶ For faster propagation next time: pre-lower TTL days before making changes (TTL pre-warm)

Follow-up Probe: What is the recommended TTL strategy for a record that may need to failover quickly, and how do you balance that with DNS infrastructure load?

Q30

Route 53

Medium

How would you migrate DNS from an on-premise DNS server to Route 53 with zero downtime for a domain serving millions of requests per day?

What they're testing: DNS migration, TTL management, validation, cutover

 Key Answer Points:

- ▶ Step 1: Audit all existing DNS records and replicate them exactly in Route 53 hosted zone
- ▶ Step 2: Set all TTLs in Route 53 to match existing (or lower to 60s for faster propagation later)
- ▶ Step 3: Add Route 53 name servers to domain registrar as secondary DNS (parallel operation)
- ▶ Step 4: Test all records from multiple regions using dig against Route 53 nameservers directly
- ▶ Step 5: Update registrar to use Route 53 nameservers as primary — cutover
- ▶ Step 6: Monitor DNS query logs and application health immediately after cutover
- ▶ Step 7: Decommission old DNS server after 48h observation period
- ▶ Critical: never delete old DNS records until propagation is confirmed complete

 **Follow-up Probe:** How would you use Route 53 Resolver and Resolver Rules to maintain DNS resolution for on-prem resources while also resolving cloud resources during a hybrid cloud migration?

CloudFront — CDN & Edge

Q31–Q32 · Interview Questions

Q31

CloudFront

Hard

Your CloudFront distribution is serving stale content even after you deployed new files to S3. How do you fix this immediately and prevent it in the future?

 **What they're testing:** CloudFront cache invalidation, cache control headers, versioning strategy

 Key Answer Points:

- ▶ Immediate fix: create a CloudFront invalidation — /* or specific paths — but costs \$0.005/path (first 1000 free/month)
- ▶ Better fix: use cache-busting file naming — myapp.v2.js instead of myapp.js
- ▶ Best practice: versioned file paths /v1.2.3/app.js — new deploy = new path = never stale
- ▶ Set correct Cache-Control headers on S3 objects: immutable for versioned assets, no-cache for HTML
- ▶ CloudFront Functions: can inject cache-busting headers at the edge
- ▶ Origin Cache-Control: S3 object metadata max-age controls how long CloudFront caches
- ▶ Use CloudFront cache policies — separate TTL for different content types (HTML vs assets)

 **Follow-up Probe:** How would you architect a CI/CD pipeline that automatically invalidates only the changed files in CloudFront rather than invalidating everything with /*?

Q32

CloudFront

Medium

How would you use CloudFront to implement geo-restriction, redirect HTTP to HTTPS, and add security headers to all responses?

 **What they're testing:** CloudFront security, Lambda@Edge, CloudFront Functions, security headers

 Key Answer Points:

- ▶ Geo-restriction: CloudFront distribution settings → Geo-restriction → block or allow specific countries
- ▶ HTTP → HTTPS redirect: in CloudFront behaviour, set 'Redirect HTTP to HTTPS' — handled at edge
- ▶ Security headers: use CloudFront Response Headers Policy — add HSTS, CSP, X-Frame-Options, etc.
- ▶ Response Headers Policy is managed in console — no code needed for standard headers
- ▶ Custom security headers: CloudFront Functions (sub-millisecond, cheapest) can add/modify headers
- ▶ Lambda@Edge: for complex logic (auth, personalisation) — runs in Lambda at edge locations
- ▶ Always use CloudFront Functions over Lambda@Edge for simple header manipulation — 6x cheaper

 **Follow-up Probe:** What is the difference between CloudFront Functions and Lambda@Edge in terms of execution environment, supported triggers, and use cases?



VPC — Virtual Private Cloud

Q33–Q35 · Interview Questions

Q33

VPC

Hard

Design a secure, highly available VPC architecture for a three-tier application (web, app, database) that follows AWS best practices.

🎯 **What they're testing:** VPC design, subnet architecture, multi-AZ, defence in depth

✅ **Key Answer Points:**

- ▶ 3 AZs minimum: public subnet (web/ALB), private subnet (app), isolated subnet (database) per AZ
- ▶ Internet Gateway for public subnets; NAT Gateway (one per AZ) for private subnet egress
- ▶ ALB in public subnets; EC2/ECS in private subnets; RDS in isolated subnets (no internet route)
- ▶ Security Groups (stateful): ALB SG → App SG → DB SG — allow only necessary ports
- ▶ NACLs (stateless): additional defence — deny known-bad CIDRs at subnet level
- ▶ VPC Flow Logs: capture all traffic for security analysis and troubleshooting
- ▶ VPC Endpoints: S3 and DynamoDB gateway endpoints — avoid NAT Gateway for AWS services
- ▶ AWS Network Firewall: managed stateful inspection for North-South and East-West traffic

💬 **Follow-up Probe:** How would you implement network micro-segmentation between microservices in the app tier that are all in the same private subnet?

Q34

 VPC

Hard

Your application needs to connect to an on-premises data centre. Compare AWS Site-to-Site VPN, AWS Direct Connect, and Transit Gateway — when would you use each?

 **What they're testing:** Hybrid connectivity, Direct Connect, VPN, Transit Gateway

 Key Answer Points:

- ▶ Site-to-Site VPN: encrypted over internet, quick to set up, up to 1.25Gbps per tunnel, good for backup
- ▶ Direct Connect: dedicated private line, 1/10/100Gbps, consistent latency, compliance-friendly, 2+ months to provision
- ▶ Direct Connect + VPN: use VPN as failover for Direct Connect (active/active or active/passive)
- ▶ Transit Gateway: hub-and-spoke to connect 100s of VPCs and on-prem — replaces VPC peering mesh
- ▶ Transit Gateway with Direct Connect Gateway: one DX connection to all VPCs across regions
- ▶ Decision: test/dev/urgent → VPN. Production/compliance/high-bandwidth → Direct Connect. Multi-VPC → Transit Gateway.
- ▶ Cost: DX has port hour + data transfer; VPN has per-connection hour + data transfer

 **Follow-up Probe:** How would you design a network architecture where 50 VPCs across 5 AWS accounts need to communicate with each other and with a single on-premises data centre?

Q35

 VPC

Medium

A DevOps engineer accidentally opened port 22 (SSH) to 0.0.0.0/0 on a production security group. What do you do immediately and how do you prevent this in future?

 **What they're testing:** Security group management, incident response, preventive controls

 Key Answer Points:

- ▶ IMMEDIATE: Remove the 0.0.0.0/0 rule from the security group via console or CLI
- ▶ Check CloudTrail: who made the change, when, from which IP/session
- ▶ Check VPC Flow Logs: was port 22 accessed from external IPs during the window?
- ▶ If accessed: treat as potential compromise — rotate credentials, check for backdoors
- ▶ Restrict SSH: only allow your office CIDR or VPN CIDR — never 0.0.0.0/0
- ▶ Better: disable SSH entirely — use AWS Systems Manager Session Manager (no open ports needed)
- ▶ Prevention: AWS Config Rule (restricted-ssh) — auto-remediates within minutes using Lambda
- ▶ AWS Security Hub: continuously checks for CIS benchmark violations including this one

 **Follow-up Probe:** How would you implement a preventive control using AWS Config + Lambda that automatically reverts any security group change that opens port 22 or 3389 to 0.0.0.0/0?

Security Groups & NACLs

Q36–Q37 · Interview Questions

Q36

 Security Groups & NACLs

Medium

Explain the difference between Security Groups and NACLs. In what scenario would you use NACLs even though Security Groups exist?

🎯 **What they're testing:** Network security fundamentals, stateful vs stateless, layered security

✅ **Key Answer Points:**

- ▶ Security Groups: stateful (return traffic automatic), instance-level, allow rules only, evaluate all rules
- ▶ NACLs: stateless (must define both inbound and outbound), subnet-level, allow AND deny rules, ordered evaluation
- ▶ Because NACLs are stateless: must allow outbound for inbound connections AND ephemeral port range (1024-65535)
- ▶ Use NACLs for: blocking specific IP ranges at subnet level (DDoS mitigation, blacklisting)
- ▶ NACLs can DENY — Security Groups cannot; this is the key operational difference
- ▶ Use case: block a specific malicious IP range across all instances in a subnet — one NACL rule
- ▶ Layered defence: Security Groups = application layer, NACLs = subnet layer
- ▶ NACLs evaluated before packets reach instances — more efficient for bulk blocking

💬 **Follow-up Probe:** If you block port 443 outbound in a NACL but the Security Group allows it, what happens? Trace the exact packet path.

Q37



Security Groups & NACLs

Hard

You need to allow an on-premises application to connect to RDS in a private subnet, but the RDS should not be accessible from the internet at all. Design the Security Group configuration.

What they're testing: Security group chaining, least privilege, hybrid network security

Key Answer Points:

- ▶ RDS Security Group: inbound rule → allow port 3306 (MySQL) from App Server SG (SG reference, not CIDR)
- ▶ App Server Security Group: inbound from ALB SG on app port; outbound to RDS SG on 3306
- ▶ For on-prem: VPN/Direct Connect terminates in VPC → on-prem CIDR allowed in App SG or separate DB SG
- ▶ DO NOT add on-prem CIDR directly to RDS SG — add to a bastion/jump host SG, then App SG → RDS SG
- ▶ RDS: place in isolated subnet (no route table entry to internet or NAT)
- ▶ Never add 0.0.0.0/0 to RDS SG outbound — scope to specific destinations
- ▶ SG references (source = another SG) are more secure than CIDR ranges — don't require IP management

Follow-up Probe: How do Security Group references (using another SG as source) work in a shared-service VPC model across multiple AWS accounts?



EKS — Elastic Kubernetes Service

Q38–Q40 · Interview Questions

Q38



EKS

Hard

Your EKS cluster nodes are running out of capacity but new nodes are not being added by the Cluster Autoscaler. How do you troubleshoot?

What they're testing: EKS Cluster Autoscaler, node groups, ASG integration, IAM

Key Answer Points:

- ▶ Check CA logs: `kubectl logs -n kube-system -l app=cluster-autoscaler` — exact error reason
- ▶ Check pending pods: `kubectl get pods --all-namespaces | grep Pending`
- ▶ CA does NOT scale if: pods have tolerations that don't match any node group, pod requests too large
- ▶ CA IAM: node group IAM role must have `autoscaling:DescribeAutoScalingGroups`, `SetDesiredCapacity`
- ▶ Node group tags: `cluster-autoscaler.kubernetes.io/enabled=true` must be on the ASG
- ▶ Max node count: check if ASG max size is already reached
- ▶ Expander algorithm: least-waste vs random vs priority — affects which node group scales
- ▶ Alternative: Karpenter (AWS-built, faster, more flexible than CA) — provision in <30s

Follow-up Probe: What is the difference between Cluster Autoscaler and Karpenter, and why is Karpenter considered superior for EKS workloads at scale?

Q39



EKS

Hard

How do you implement IAM roles for pods in EKS (IRSA) and why is it better than assigning IAM

roles to nodes?

🎯 **What they're testing:** IRSA, OIDC, pod-level IAM, least privilege

✅ **Key Answer Points:**

- ▶ Node-level IAM: all pods on the node share the same IAM role — violates least privilege
- ▶ IRSA (IAM Roles for Service Accounts): pod gets its own IAM role via OIDC federation
- ▶ How it works: EKS OIDC provider → Kubernetes ServiceAccount annotation → IAM role trust policy
- ▶ Pod's ServiceAccount annotated with: `eks.amazonaws.com/role-arn: arn:aws:iam::<id>:role/<role>`
- ▶ IAM role trust policy: allows OIDC provider to assume role for specific ServiceAccount
- ▶ AWS SDK in pod automatically gets temporary credentials via web identity token
- ▶ Pod Identities (newer): simpler than IRSA — no OIDC setup needed, managed by EKS Add-on
- ▶ Best practice: one IAM role per workload with minimal required permissions

💬 **Follow-up Probe:** Walk me through creating IRSA for a pod that needs to read from an S3 bucket — list every step from IAM to pod deployment.

Q40

 EKS

Medium

An EKS cluster upgrade is needed from 1.28 to 1.30. Describe your upgrade strategy to avoid downtime.

 **What they're testing:** EKS upgrade strategy, version skew, worker node management

 Key Answer Points:

- ▶ EKS supports N-2 version skew between control plane and nodes — upgrade incrementally (1.28 → 1.29 → 1.30)
- ▶ Step 1: Review EKS release notes and Kubernetes changelog for breaking changes
- ▶ Step 2: Test in dev/staging cluster first
- ▶ Step 3: Update add-ons (CoreDNS, kube-proxy, VPC CNI) to versions compatible with new K8s version
- ▶ Step 4: Upgrade control plane via EKS console/CLI (managed upgrade, no downtime for control plane)
- ▶ Step 5: Upgrade managed node groups — EKS drains and replaces nodes in rolling fashion
- ▶ Step 6: If self-managed nodes: cordon, drain, terminate, new nodes launch with new AMI
- ▶ Step 7: Validate: kubectl get nodes, run smoke tests, check all workloads healthy

 **Follow-up Probe:** What is the risk of having a cluster where the control plane is on 1.30 but some node groups are still on 1.27, and how does Kubernetes handle version skew?



IAM & Security

Q41–Q42 · Interview Questions

Q41

 IAM & Security

Hard

Your security audit found IAM users with AdministratorAccess and long-lived access keys in production. How do you remediate this without breaking anything?

 **What they're testing:** IAM hardening, least privilege, credential rotation, break-glass

 Key Answer Points:

- ▶ Step 1: Identify all access key usage — AWS CloudTrail + IAM Last Used metadata
- ▶ Step 2: For inactive keys (not used 90+ days): immediately deactivate (not delete) — observe 2 weeks
- ▶ Step 3: Replace human users with IAM Identity Center (SSO) — no long-lived keys
- ▶ Step 4: Replace application access keys with IAM roles (instance profiles, IRSA, task roles)
- ▶ Step 5: Remove AdministratorAccess — implement least privilege per role/team
- ▶ Step 6: Enable MFA on all human IAM users/roles — require in SCP
- ▶ Break-glass: one locked emergency admin user with MFA, credentials in sealed physical vault
- ▶ AWS Organizations SCP: prevent creation of IAM users or access keys in member accounts

 **Follow-up Probe:** How would you use AWS Organizations Service Control Policies to prevent any IAM user from ever having AdministratorAccess across your entire organisation?

Q42

 IAM & Security

Medium

Describe how you would implement cross-account access in AWS — for example, allowing a deployment pipeline in Account A to deploy to Account B.

 **What they're testing:** Cross-account IAM roles, STS AssumeRole, trust relationships

 Key Answer Points:

- ▶ Account B: create IAM role with permission for deployment (e.g., ECS or EC2 deploy actions)
- ▶ Trust policy on the role: allow Account A's IAM role/user to sts:AssumeRole
- ▶ Account A: pipeline role has sts:AssumeRole permission for Account B role ARN
- ▶ Pipeline assumes Account B role: `aws sts assume-role` → gets temporary credentials
- ▶ Use temporary credentials for deployment actions — expire in 1 hour max
- ▶ For CI/CD: CodePipeline supports cross-account with assumed roles natively
- ▶ External ID: add `ExternalId` condition in trust policy to prevent confused deputy attacks
- ▶ Audit: CloudTrail in Account B shows AssumeRole events with source Account A identity

 **Follow-up Probe:** What is the confused deputy problem in AWS, and how does the `ExternalId` condition in an IAM trust policy prevent it?



CloudWatch & Observability

Q43–Q44 · Interview Questions

Q43



CloudWatch

Hard

You need to implement centralised logging and monitoring for 50 AWS accounts across 3 regions. Design the architecture.

What they're testing: Multi-account logging, CloudWatch cross-account, centralised architecture

Key Answer Points:

- ▶ Use AWS Organizations with a dedicated Log Archive account
- ▶ CloudWatch cross-account observability: share metrics, logs, traces from all accounts to central account
- ▶ S3 centralised log archive: all accounts push CloudTrail, VPC Flow Logs, ALB logs to central S3 bucket
- ▶ EventBridge cross-account: events from all accounts forwarded to central event bus
- ▶ AWS Security Hub + GuardDuty: findings from all accounts aggregated in management account
- ▶ Use IAM roles in each account to allow central account to read CloudWatch data
- ▶ Amazon OpenSearch Service + Firehose: real-time log analytics across all accounts
- ▶ Cost optimisation: use CloudWatch Logs subscriptions to filter before sending to S3 — avoid storing all logs in CW

Follow-up Probe: How would you design a tiered log retention strategy — hot logs in CloudWatch Logs, warm logs in S3 Standard, cold logs in Glacier — with automatic transitions?

Q44



CloudWatch

Medium

Your team wants to be alerted on business metrics (orders per minute, checkout errors) not just infrastructure metrics. How do you implement this in AWS?

What they're testing: Custom CloudWatch metrics, embedded metric format, application observability

Key Answer Points:

- ▶ CloudWatch custom metrics: application pushes business metrics via PutMetricData API
- ▶ Embedded Metric Format (EMF): structured JSON in CloudWatch Logs auto-extracted as metrics — no API calls
- ▶ Lambda / ECS / EC2: use AWS Embedded Metric Format SDK to emit metrics in logs
- ▶ Metric math: create 'error rate = errors/total_requests * 100' as a computed metric
- ▶ CloudWatch Alarms on custom metrics: orders < 100/min for 5 minutes → PagerDuty/SNS
- ▶ CloudWatch Dashboards: business metrics + infrastructure metrics on one screen
- ▶ X-Ray + CloudWatch ServiceLens: trace individual transactions and see business impact
- ▶ EventBridge + Lambda: complex anomaly detection logic beyond what alarms support

Follow-up Probe: How would you implement anomaly detection for a metric that has different baseline values during business hours vs nights/weekends?



Cost Optimisation

Q45–Q46 · Interview Questions

Q45

 Cost Optimisation

Hard

Your AWS bill suddenly increased by 40% this month. How would you identify the cause and implement controls to prevent unexpected cost spikes?

 **What they're testing:** Cost investigation, AWS Cost Explorer, budget alerts, tagging

 Key Answer Points:

- ▶ AWS Cost Explorer: drill down by service, linked account, region, tag — find the spike
- ▶ Cost and Usage Report (CUR): most granular — hourly, by resource ID
- ▶ AWS Cost Anomaly Detection: ML-based, automatically alerts on unusual spend patterns
- ▶ Common causes: NAT Gateway data transfer surge, EC2 On-Demand spike, S3 GET request surge, data transfer
 - ▶ Tagging strategy: enforce cost allocation tags (team, environment, project) via SCP to enable cost attribution
- ▶ AWS Budgets: set monthly budget with SNS alert at 80% and 100% of threshold
- ▶ Reserved Instance/Savings Plans coverage: low coverage = high On-Demand spend
- ▶ Prevention: AWS Budgets with action — automatically apply SCP or stop EC2 when budget exceeded

 **Follow-up Probe:** How would you implement an automated cost governance system where spending in non-production accounts is automatically capped and resources are stopped after business hours?

Q46

 Cost Optimisation

Medium

Your company wants to reduce AWS costs for idle and underutilised resources. What strategies and tools would you implement?

 **What they're testing:** Cost optimisation, rightsizing, waste identification

 Key Answer Points:

- ▶ AWS Compute Optimizer: identifies over-provisioned EC2, Lambda, ECS, EBS — gives right-size recommendations
- ▶ EC2 rightsizing: move from m5.xlarge to m5.large if CPU < 20% consistently
- ▶ Idle resources: EC2 instances with < 5% CPU over 2 weeks, unattached EBS volumes, unused Elastic IPs
- ▶ Savings Plans / Reserved Instances: 30-72% savings for predictable workloads
- ▶ S3: Intelligent-Tiering, delete incomplete multipart uploads, lifecycle to Glacier
- ▶ RDS: stop dev/test databases on schedule (Start/Stop RDS); use Aurora Serverless for variable workloads
- ▶ Data Transfer: use VPC Endpoints to avoid NAT Gateway for S3/DynamoDB traffic
- ▶ Scheduled auto-scaling: scale in app to 0 or 1 instance overnight for non-production

 **Follow-up Probe:** What is the typical cost reduction achievable from a well-executed AWS cost optimisation programme, and how would you build a FinOps culture in an engineering team?



DevOps, IaC & CI/CD

Q47–Q48 · Interview Questions

Q47

 DevOps & IaC

Hard

You need to manage infrastructure for 20 microservices across 4 AWS environments (dev/staging/prod-eu/prod-us) using Terraform. How do you structure this to avoid duplication and drift?

 **What they're testing:** Terraform at scale, module design, remote state, workspace vs directory structure

 Key Answer Points:

- ▶ Module strategy: reusable modules in a modules/ directory (ecs_service, rds, alb, etc.)
- ▶ Environment directories: environments/dev/, environments/staging/, environments/prod-eu/ — each has main.tf calling modules
- ▶ Remote state: Terraform Cloud or S3 + DynamoDB locking — separate state file per environment
- ▶ Variables: environment-specific tfvars files — dev.tfvars has smaller instance types, lower counts
- ▶ Avoid Terraform workspaces for environment separation — directory isolation is cleaner and safer
- ▶ CI/CD for Terraform: Atlantis or Terraform Cloud runs plan on PR, apply on merge
- ▶ Drift detection: terraform plan in CI on schedule — alert on drift
- ▶ Policy as code: Sentinel or OPA Conftest — enforce naming, tagging, instance type restrictions

 **Follow-up Probe:** How would you handle the situation where the production Terraform state gets corrupted or lost, and what preventive controls would you implement?

Q48

 DevOps & IaC

Medium

Describe how you would implement a complete CI/CD pipeline for a containerised microservice on AWS — from code commit to production with automated rollback.

 **What they're testing:** CI/CD pipeline design, CodePipeline/GitHub Actions, ECS/EKS deployment, rollback

 Key Answer Points:

- ▶ Source: GitHub/CodeCommit → triggers pipeline on push to main
- ▶ Build: CodeBuild/GitHub Actions — runs tests, builds Docker image, pushes to ECR with git SHA tag
- ▶ Security scan: ECR image scanning or Trivy in pipeline — fail on critical CVEs
- ▶ Staging deploy: ECS/EKS rolling deploy to staging; run integration/smoke tests
- ▶ Production deploy: manual approval gate → deploy to prod with maxUnavailable:0, maxSurge:1
- ▶ Monitoring gate: watch error rate, latency for 10 minutes post-deploy
- ▶ Automatic rollback: if CloudWatch alarm triggers → Lambda calls ECS/EKS rollback API
- ▶ Notifications: Slack/PagerDuty at each stage — start, success, failure, rollback

 **Follow-up Probe:** How would you implement feature flags in this pipeline so a feature can be deployed to production but only enabled for 1% of users without a re-deployment?



Additional Services (SQS/SNS/EventBridge/Secrets)

Q49–Q50 · Interview Questions

Q49



SQS / SNS / EventBridge

Hard

Compare SQS, SNS, and EventBridge. For a microservices event-driven architecture with fan-out, filtering, and replay requirements — which would you choose and why?

What they're testing: Event-driven architecture, messaging services, trade-offs

Key Answer Points:

- ▶ SQS: point-to-point queuing, guaranteed delivery, retention 14 days, consumer pulls — best for task queues and worker patterns
- ▶ SQS FIFO: ordered, exactly-once processing, 3000 msg/s with batching — best for ordered workflows
- ▶ SNS: pub/sub fan-out to multiple subscribers (SQS, Lambda, HTTP, email) — no retention, fire-and-forget
- ▶ SNS + SQS pattern: SNS fans out to multiple SQS queues — each consumer has its own queue with retry/DLQ
- ▶ EventBridge: event bus with rich filtering/routing rules, Schema Registry, 100+ AWS service integrations
- ▶ EventBridge Replay: archive and replay all events — best for event sourcing and debugging
- ▶ EventBridge Pipes: filter → enrich → target pipeline with minimal code
- ▶ Decision: internal AWS event routing → EventBridge. Worker queue → SQS. Notification fan-out → SNS+SQS.

Follow-up Probe: Design an event-driven system where an order placement event must trigger: inventory update, email confirmation, fraud check, and analytics — each independently and with different retry strategies.

Q50



AWS Secrets Manager & Systems Manager

Hard

Design a zero-trust secrets management strategy for an organisation with 100 microservices, each needing different secrets, running across EC2, Lambda, and EKS.

What they're testing: Secrets management at scale, least privilege, rotation, zero-trust

Key Answer Points:

- ▶ Separate secrets per service per environment: /myorg/<env>/<service>/db-password
- ▶ IAM least privilege: each service's IAM role ONLY allows GetSecretValue on its own secret ARN
- ▶ EC2: IAM instance profile → Secrets Manager. Lambda: execution role. EKS: IRSA per service.
- ▶ EKS: External Secrets Operator syncs Secrets Manager → Kubernetes Secrets automatically
- ▶ Rotation: Secrets Manager automatic rotation for RDS, Redshift, DocumentDB — no code changes
- ▶ Custom rotation Lambda for non-AWS services (third-party APIs, SaaS credentials)
- ▶ Audit: CloudTrail logs every GetSecretValue — alert on unusual access patterns
- ▶ Parameter Store SecureString: for configuration (non-secret) values — free tier, 4KB limit
- ▶ Secrets Manager resource policy: add deny condition for specific IP ranges or VPCs
- ▶ Centralised secrets account: secrets live in one account, cross-account role access — simplifies rotation and audit

Follow-up Probe: How would you detect and respond to a secret being accessed from an unexpected IP address or region in real-time using AWS-native services?

QUICK REFERENCE — AWS SERVICES CHEAT SHEET

Service	Primary Use Case	Key Limits / Facts	Common Gotcha
EC2	Virtual servers	1000s of instance types; User data max 16KB	User data runs once — use cloud-init for re-run
ALB	Layer 7 HTTP/S load balancing	10 rules/listener; WAF attachment	ALB SG must be opened in target SG (not by CIDR)
Auto Scaling	Horizontal EC2 scaling	Cooldown 300s default; 1 ASG → multiple AZs	Warmup period: new instance metrics not counted during startup
RDS	Managed relational database	Max 40TB GP2; Multi-AZ = sync replica, HA	Read replicas are async — can lag; PITR needs automated backups enabled
S3	Object storage	Objects up to 5TB; bucket names globally unique	Eventually consistent for overwrite; S3 versioning per object
Lambda	Serverless compute	15 min max; 10GB memory; 1000 concurrency default	VPC Lambda: cold start +1s; init code cached between invocations
Route 53	DNS + health checks	60s min TTL practical; health check interval 10/30s	TTL pre-lower days before changes; negative cache from NXDOMAIN
CloudFront	CDN + edge compute	450 PoPs; default TTL 24h; invalidations cost \$0.005	OAC replaces OAI; CF Functions cheaper than Lambda@Edge
VPC	Private network	/16 max CIDR; 200 subnets/VPC default	NACL is stateless — must allow ephemeral ports outbound (1024-65535)
Security Groups	Stateful instance firewall	60 inbound + 60 outbound rules default	SG rules are allow-only; use NACL for deny
EKS	Managed Kubernetes	Up to 450 nodes/cluster; Fargate or EC2 nodes	IRSA requires OIDC provider setup; Karpenter faster than Cluster Autoscaler
IAM	Identity and access management	Session tokens expire 1h default (STS)	Deny always overrides Allow; SCPs apply before IAM policies
SQS	Message queue	14-day retention; 256KB max message; FIFO = 3K/s	Visibility timeout must be > Lambda execution time to avoid duplicate processing
Secrets Manager	Secrets storage + rotation	Costs \$0.40/secret/month; free API calls in Lambda	Retrieve outside handler for caching; rotation needs Lambda in same VPC as DB